

WEEK-1,2

WEEK-1

🔥 1. Software ko “components” me sochna

👉 PDF me bhi diya hai ki software ko **chhote-chhote parts (modules)** me todte hain

📌 Example: Amazon App

Socho Amazon ek hi cheez nahi hai — usme multiple parts hote hain:

- 🛒 Cart system
- 💳 Payment system
- 📦 Inventory system
- 🔍 Search system

👉 Har ek alag component hai

🎯 Simple samajh:

Software = Lego blocks 🧱

Har block alag kaam karta hai, milke system banata hai

🔥 2. First Step: Requirement Specification

👉 Sabse pehla step: “User ko kya chahiye?”

📌 Example 1: Food Delivery App

Agar tum Zomato jaisa app bana rahe ho:

❌ Galti kya hoti hai?

Direct coding start kar di

✅ Sahi approach:

Pehle decide karo:

- User kya karega?
 - Restaurant kaise dikhega?
 - Payment kaise hoga?
-

📌 Example 2: Mobile Banking App

PDF me bhi diya hai:

- User login kare
- Balance check kare
- Money transfer kare

👉 Ye sab **requirements** hain

📌 Clients ka concept (important 🔥)

3 type ke clients:

1. External User

👉 Tum aur main (bank app use karte hain)

2. Internal Client

👉 Company ke employees (HR portal)

3. Another Software

👉 Ek app dusre app se baat karta hai
Example: Payment gateway (Razorpay)

3. Design Phase (Big Picture)

👉 Ab system ka **structure decide** karte hain

PDF me likha hai:

Design gives big picture view

Example:

Tum ek app bana rahe ho:

Design me decide hoga:

- Database kaha hoga?
- Backend kaise chalega?
- Frontend kya hoga?

👉 Jaise ghar banane se pehle map banate hain 🏠

4. Development (Coding Phase)

👉 Ab actual coding hoti hai

PDF ke according:

- Code likha jata hai
 - Different developers kaam karte hain
-

Example:

Team me 3 log:

- 🧑 A → Login system
- 🧑 B → Payment system
- 🧑 C → UI

👉 Sab alag kaam karte hain

⚠️ Problem kya hoti hai?

PDF me likha:

Different developers ka idea alag ho sakta hai

👉 Result:

- Code clash
 - Integration issues
-

🔥 5. Testing (Most Important 🚒)

👉 Software ko check karte hain

PDF:

Ensure software works as expected

📌 Example:

Tumne payment app banaya:

Testing ke bina:

- ₹100 bheja → ₹1000 deduct 😱
-

Types:

- Unit Testing → small part check
- Integration Testing → parts together
- Acceptance Testing → user check

Alpha vs Beta:

- Alpha → company ke andar test
 - Beta → real users test
-

6. Maintenance (After Launch)

👉 App ban gaya = kaam khatam? ❌

PDF ke according:

- Updates
 - Bug fixes
 - New features
-



Example:

Ashish Maurya

Instagram:

- Reels add hua
 - Dark mode aaya
 - Bugs fix hote rehte hain
-

Final Summary (Exam Ready 100)

पूरा flow yaad rakho:

1. Requirements → kya banana hai
 2. Design → kaise banana hai
 3. Development → code likho
 4. Testing → check karo
 5. Maintenance → update karo
-



Ekदम solid example (Full Flow)

👉 Tum ek **college app** bana rahe ho:

1. Requirement:
 - Attendance
 - Marks
 - Notice
 2. Design:
 - Database + UI plan
 3. Development:
 - Coding
 4. Testing:
 - Bugs check
 5. Maintenance:
 - New features (chat, alerts)
-



Golden Line (Yaad rakhna)

“Direct coding = biggest mistake in Software Engineering”

Ashish Maurya



1.6 Waterfall Model (Plan & Document Model)



Concept:

- 👉 Sab kuch **step-by-step sequential order** me hota hai
 - 👉 Ek phase khatam → tabhi next phase start
-



Flow:

Requirements → Design → Development → Testing → Maintenance

- 👉 Ek waterfall (paani girna) jaisa flow — **peeche wapas nahi ja sakte**

Real-Life Example:

Ghar banana

1. Pehle pura plan (map)
2. Fir design
3. Fir construction
4. Fir finishing

👉 Beech me change karna mushkil ❌

Problems:

- ❌ Changes allow nahi hote easily
 - ❌ Late testing → bugs late milte hain
 - ❌ Real-world me requirements change hote rehte hain
-

When to use:

- Jab requirements **clear & fixed** ho
 - Government / banking type projects
-

Exam Trick Line:

“Waterfall is rigid, sequential, and documentation-heavy”

1.7 Agile Model (Modern Approach)

Concept:

- 👉 Software ko **chhote parts (iterations)** me banate hain
 - 👉 Har step me **feedback + improvement**
-



Flow:

Plan → Build → Test → Feedback → Repeat 🔄



Real-Life Example:



Instagram App

- Pehle basic version launch
- Fir update: reels
- Fir update: stories
- Fir update: UI

👉 Continuous improvement 🔥



Key Features:

- ✅ Flexible
 - ✅ Fast updates
 - ✅ Continuous feedback
 - ✅ User involvement
-



Problems:

- ❌ Proper planning kabhi missing hota hai
 - ❌ Large systems me manage karna tough
-



Waterfall vs Agile (IMPORTANT 🔥)

Feature	Waterfall ❌	Agile ✅
---------	-------------	---------

Flexibility	Low	High
Changes	Difficult	Easy
Testing	End me	Har step
Delivery	Late	Continuous
Approach	Sequential	Iterative

Simple Line (Yaad rakhna)

👉 Waterfall = “Pehle sab plan karo, fir build karo”

👉 Agile = “Thoda build karo, feedback lo, improve karo”

Final Example (Solid Understanding)

E-commerce App

Waterfall:

- 6 months planning
- 6 months coding
- Last me launch

👉 Risk high ❌

Agile:

- Week 1: login
- Week 2: cart
- Week 3: payment
- Har week feedback

👉 Safe + fast ✅

Bonus Trick Insight (Exam Gold 100)

👉 Agar question me keywords aaye:

- “fixed requirements” → Waterfall
- “frequent change / feedback” → Agile
- “incremental delivery” → Agile
- “strict process / documentation” → Waterfall
-

[Test yourself-1](#)

[Test yourself-2](#)

WEEK-2

Ashish Maurya

1. Requirement Gathering & Analysis (CORE FOUNDATION)

📄 (Page 4) → Agar requirement galat → cost ↑ + rework ↑

Concept (Deep Understanding)

Software banane se pehle sabse bada question:

👉 “Exactly kya banana hai?”

Agar ye clear nahi → baaki sab waste

⚠️ Why important?

1. Developer ko clarity milti hai
 2. Customer aur developer agree karte hain
 3. Future rework avoid hota hai
-

❌ Real Problem (Exam Favorite)

👉 Agar requirement unclear:

- Ambiguity (same baat ka multiple meaning)
 - Incomplete requirement
 - Conflicting requirement
-

📌 Deep Example:

Amazon Seller Portal:

Customer bolta hai:

👉 “Mujhe fast system chahiye”

Problem:

- Fast = 1 sec? 5 sec? 10 sec?

👉 Ye non-measurable requirement hai ❌

🔥 2. Types of Users (VERY IMPORTANT)

📄 (Page 6–8)

🧠 Why needed?

👉 Har user ka requirement alag hota hai

◆ 1. Primary Users

👉 Direct interaction

📌 Example:

- Seller login karta hai
- Product upload karta hai

👉 **System design directly inke liye hota hai**

◆ 2. Secondary Users

👉 Direct use nahi, but output use karte hain

📌 Example:

- Manager reports dekhta hai
-

◆ 3. Tertiary Users

👉 System use nahi karte but impact hota hai

📌 Example:

- Government (tax rules)
 - Competitors
-

🔥 Exam Trap:

👉 Students confuse:

- Secondary ≠ indirect user always
 - Tertiary = “affected but not user”
-

3. Requirement Gathering Techniques (DEEP)

 (Page 15–24)

WHY multiple techniques?

 Ek technique se **complete picture nahi milti**

1. Questionnaires

✓ When to use:

- Large audience
- Structured data

✗ Limitation:

- Deep understanding nahi milti

Example:

100 sellers ko Google form bhejna

2. Interviews

✓ Types:

- Structured (fixed questions)
- Unstructured (free flow)
- Semi-structured (mix)

✓ Best for:

- Deep insights

Limitation:

- Time consuming
-

3. Focus Groups

✓ Why:

- Multiple opinions ek saath

Example:

Mobile vs Furniture sellers

 Conflict bhi detect hota hai

4. Naturalistic Observation

✓ Concept:

 “Real world me dekhna”

Example:

Shopkeeper ka behaviour observe karna

5. Documentation

✓ Use:

- Rules samajhne ke liye

Example:

Bank regulations

Exam Insight:

- 👉 Question: "Which technique gives real-world behavior?"
👉 Answer: **Observation**
-

🔥 4. Requirement Analysis

📄 (Page 11)

🧠 Purpose:

👉 Identify:

- Ambiguity
- Inconsistency
- Missing requirements

🧠 1. Ambiguity (अस्पष्टता)

👉 Jab requirement **clear nahi hoti / multiple meanings nikal sakte hain**

🔍 Example:

"System should load fast"

❌ Problem:

- "fast" ka matlab kya? 1 sec? 5 sec?

✅ Fix:

"System should load within 2 seconds"

👉 **Rule:** Har requirement measurable honi chahiye

⚠️ 2. Inconsistency (विरोधाभास)

👉 Jab do requirements **ek dusre se clash karte hain**

Example:

- Req 1: "Password must be exactly 8 characters"
- Req 2: "Password length should be at least 10 characters"

✗ Dono ek saath possible nahi

👉 Ye **logical conflict** hai

✓ Fix:

- Clarify with stakeholder
 - Ek consistent rule define karo
-

? 3. Missing Requirements (अधूरी requirements)

👉 Jab kuch **important cheezein batayi hi nahi gayi**

Example:

"User can login using email"

✗ Missing:

- Password ka kya?
 - Forgot password?
 - Email verification?
-

Example:

Requirement:

👉 "User can search product"

Missing:

- By name?
- By category?
- By price?

👉 → Incomplete ❌

🔥 5. Functional vs Non-Functional (VERY IMPORTANT 🔥)

📄 (Page 27–37)

◆ Functional Requirements (FR)

👉 System kya karega

📌 Examples:

- Add product
 - Delete product
 - Track orders
- Ashish Maurya*
-

◆ Non-Functional Requirements (NFR)

👉 System kaise behave karega

📌 Examples:

- Speed
 - Security
 - Reliability
-

🧠 Deep Difference:

Aspect	Functional	Non-Functional
Focus	Action	Quality

Example Add product Load < 3 sec

Advanced Insight:

👉 FR = necessary

👉 NFR = differentiator

👉 Same app:

- FR same ho sakta hai
 - NFR decide karega “better app kaun sa hai”
-

Important NFR Types:

1. Reliability

👉 Same input → same output

2. Robustness

👉 Error handle kare

📌 Example:

Invalid input diya → crash nahi hona chahiye

3. Performance

👉 Speed

4. Security

👉 Data safe

6. SRS (Software Requirement Specification)

 (Page 40–48)

Concept:

 “Full system ka blueprint document”

Why needed?

- Agreement between client & developer
 - Cost estimation
 - Future reference
-

Structure (Important)

1. Introduction
 2. Overall description
 3. Specific requirements
-

Inside Specific Requirements:

- Functional
 - Non-functional
 - Interfaces
-

Example:

Seller Portal SRS:

- Add product
 - Track payment
 - Show reports
-

Problem:

- 👉 Too much documentation
 - 👉 Agile me isko avoid karte hain
-

7. External Interfaces

 (Page 44)

Types:

Ashish Maurya

1. User Interface (UI)

- 👉 App screen
-

2. Hardware Interface

- 👉 Devices
-

3. Software Interface

- 👉 APIs
-

4. Communication Interface

- 👉 Network

Example:

UPI Payment:

- UI → App
 - Software → Bank API
 - Communication → Internet
-

8. Behaviour Driven Design (BDD)

 (Page 52)

Concept:

- 👉 Focus on **behavior** (user kya karega)
 - 👉 Requirements continuously update hote hain
-

Why needed?

- 👉 Jab requirements clear nahi ho
-

9. User Stories (SUPER IMPORTANT)

 (Page 53)

Format:

As a [user]
I want [feature]
So that [benefit]

Example:

As a seller
I want to track inventory
So that I avoid stockout

Deep Insight:

👉 Ye “WHY” bhi capture karta hai

10. Benefits of User Stories

- Lightweight
 - Easy
 - Communication improve
-

11. SMART User Stories

 (Page 58)

Breakdown:

S → Specific

Clear hona chahiye

M → Measurable

Quantifiable

A → Achievable

Possible hona chahiye

R → Relevant

Business value hona chahiye

T → Timeboxed

Time limit ho

Ashish Maurya

Example:

 Bad:
“App fast hona chahiye”

 Good:
“Response time < 2 sec”

12. Drawbacks of User Stories

 (Page 65)

- Continuous user interaction needed
- Large systems me difficult



FINAL MASTER SUMMARY

- 👉 Requirement galat → project fail
- 👉 Users 3 types → sabka need important
- 👉 FR = WHAT
- 👉 NFR = HOW
- 👉 SRS = document
- 👉 Agile → User stories
- 👉 SMART → quality check

- 👉 “fast / secure / reliable” → **Non-functional**

- 👉 “add / delete / track” → **Functional**

- 👉 “real behavior” → **Observation**

- 👉 “conflict” → **Focus group**

- 👉 “unclear requirements” → **Agile**

Ashish Maurya

[Test yourself](#)

WEEK-7

WEEK-8

Ashish Maurya

WEEK-3

WEEK-3

1. Interaction Design kya hota hai?

👉 Simple:

User + System ke beech interaction ka design

Example:

- Amazon app use karte ho
- Product search → add to cart → payment

👉 Ye pura flow = **Interaction Design**

2. Interaction Design ke Steps (VERY IMPORTANT)

PDF me diya hai 👉

1. **Identify needs**
 2. **Design alternatives**
 3. **Build prototype**
 4. **Evaluate**
-

Example (Real Life)

Tu ek **Food delivery app** bana raha hai:

- Need → log jaldi food order kare
- Design → multiple UI ideas
- Prototype → dummy app
- Evaluate → users se feedback

👉 Ye cycle repeat hota hai 

3. Bad Design Example (IMPORTANT)

PDF me ek messy website dikhayi hai

👉 Problem:

- Too much content
- No structure
- Confusing

👉 Result:

✗ User frustrate ho jata hai

Exam Insight:

👉 “User frustration” = poor UI design

4. Usability kya hota hai? (MOST IMPORTANT)

Definition (ISO standard):

👉 System kitna:

- Effective
- Efficient
- Satisfying

5. Usability Goals (IITM FAVORITE 100)

1. Effectiveness

👉 System kaam sahi karta hai ya nahi

📌 Example:

- Google search → correct result deta hai ✓
-

2. Efficiency

👉 Kaam kitni jaldi hota hai

📌 Example:

- Amazon → 1-click order

👉 Kam steps = high efficiency

3. Safety

👉 User ko galti se bachana

📌 Example:

- Undo (Ctrl + Z)
- Delete → Trash

👉 Accident se recovery possible ✓



Real Case (VERY IMPORTANT)

👉 Therac-25 machine (PDF me)

- Radiation machine error
- UI design problem
- Patients harmed 😬

- 👉 Lesson:
 - 👉 **Bad UI = dangerous**
-

4. Learnability

- 👉 System seekhna kitna easy hai

📌 Example:

- WhatsApp → koi bhi use kar sakta

- 👉 Easy start = good learnability
-

5. Memorability

- 👉 Ek baar seekhne ke baad yaad rehna

📌 Example:

- MS Word tools
- Instagram UI

- 👉 Gap ke baad bhi use kar pao ✓
-

6. User Experience (UX)

- 👉 Sirf kaam nahi — **feelings** bhi important

📌 **Examples:**

- Fun 😄
- Satisfying 😊
- Attractive 😍

- 👉 Ye subjective hota hai
-

Difference (Exam me aata hai)

Usability	UX
Objective	Subjective
Task completion	Feelings
Efficiency	Emotions

7. Prototyping (VERY IMPORTANT)

👉 Coding se pehle **dummy version** banana

? Why not directly coding?

PDF me diya hai 👉

- Ideas galat ho sakte
- Time & money waste
- Testing possible nahi

👉 Isliye:

✓ Prototype → test → improve

8. Types of Prototypes

1. Storyboard

👉 Comic jaisa diagram

3 parts:

- Setting

- Sequence
- Satisfaction

📌 Example:

Food app:

- User hungry
 - App open
 - Order
 - Happy 😊
-

📦 2. Paper Prototype

👉 Hand-drawn UI

📌 Example:

- Paper pe app design
- Buttons draw karna

👉 Cheap + fast ✓

Ashish Maurya

🎯 Benefits of Prototypes

- Fast testing
 - Low cost
 - User feedback
 - Better decisions
-

🔥 FINAL REVISION (Short 100)

👉 Interaction Design = user + system flow

👉 Usability = effectiveness + efficiency + safety + learnability + memorability

👉 UX = emotions

👉 Prototype = before coding testing

IITM TRAPS (Important)

- Efficiency ≠ Effectiveness
 - Usability ≠ UX
 - Prototype ≠ Final product
 - Safety includes **error recovery**
-

18. Design Heuristics kya hote hain?

👉 Simple:

Rules / guidelines for good UI design

👉 Like:

- “User confuse nahi hona chahiye”
 - “System clear hona chahiye”
-

Real meaning:

👉 Without coding → UI ko evaluate karna

3 Types (PDF ke according)

1. Heuristics for Understanding
 2. Heuristics for Action
 3. Heuristics for Feedback
-



19. Heuristics for Understanding

👉 User ko system samajhna easy hona chahiye



Key Ideas:

1. Visibility of system status

👉 System kya kar raha hai → user ko pata hona chahiye

📌 Example:

- Loading spinner
 - “Processing...”
-

2. Match with real world

👉 System real-life jaisa hona chahiye

📌 Example:

- “Cart” icon 🛒 (shopping app)
-

3. Consistency

👉 Same design everywhere

📌 Example:

- Same button style
-



Trap:

👉 “User confused ho raha hai” → Understanding problem



20. Heuristics for Action

👉 User easily actions perform kar sake



Key Ideas:

1. User control & freedom

👉 Undo / back option hona chahiye

📌 Example:

- Undo button
-

2. Flexibility & efficiency

👉 Fast users ke liye shortcuts

📌 Example:

- Ctrl + C
-

3. Error prevention

👉 Galti hone se pehle rokna

📌 Example:

- Confirm before delete
-



Trap:

👉 “User stuck ho gaya” → Action problem



21. Heuristics for Feedback

👉 System user ko response de



Key Ideas:

1. Feedback after action

👉 Click kiya → kuch hona chahiye

📌 Example:

- Button color change
 - “Order placed”
-

2. Error messages

👉 Clear + helpful

📌 Example:

✗ “Error 404”

✓ “Page not found”

3. Help & documentation

👉 User ko guide karo



Trap:

👉 “User ko pata hi nahi kya hua” → Feedback issue

Full Comparison (VERY IMPORTANT)



Type	Focus
Understanding	System samajhna
Action	Kaam karna
Feedback	Response milna

Real Life Example (Combined)

👉 Suppose tu Amazon use kar raha hai:

- Product dikh raha → Understanding ✓
- Add to cart button → Action ✓
- “Added to cart” message → Feedback ✓

IITM MEGA TRAPS

1. “System visible nahi” → Understanding
 2. “User action nahi kar pa raha” → Action
 3. “Response nahi mil raha” → Feedback
-



FINAL SHORT REVISION

- 👉 Understanding = samajhna
- 👉 Action = karna
- 👉 Feedback = system ka jawab

[Test yourself](#)

[Test yourself -2](#)

Ashish Maurya

WEEK-4

WEEK-4

1. SOFTWARE PROJECT MANAGEMENT — CORE IDEA

? Project Management actually hota kya hai?

👉 Simple:

“Project ko time + cost + quality constraints ke andar complete karna”

3 Major Constraints (Hidden concept)

- Time (deadline)
- Cost (budget)
- Scope (features)

👉 Inko bolte hain: **Triple Constraint**

 Trap:

👉 Ek badha → dusra impact hoga

PROJECT MANAGER — REAL ROLE

Deep Understanding:

1. Communication Hub

👉 Client ↔ Developers

💡 Example:

Client: “Payment feature chahiye”

Dev: “Stripe use kare?”

👉 Manager translate karega

2. Team Formation

👉 Sirf log nahi, **right skill mix**

💀 Trap:

- 10 average dev < 3 expert dev

3. Estimation + Scheduling

👉 Ye MOST critical hai

💡 Agar galat estimate:

- Delay
- Cost overrun
- Client unhappy

4. Risk Management

👉 Future problems ko pehle sochna

2. PROJECT ESTIMATION — DEEP

? Estimation kya hai?

👉 Future ka prediction (time + cost + effort)

Key Parameters (Page 9 deep logic)

1. SIZE (KLOC)

👉 Problem:

- LOC estimate karna hi tough hai

💀 Trap:

👉 Early stage me LOC unknown hota hai

2. EFFORT (Person-Month)

👉 Definition:

1 person ka 1 month ka kaam

⚠️ VERY IMPORTANT CONCEPT

👉 Effort $\neq$ Time

💡 Example:

Project = 12 PM

Dev	Time
1	12 months
2	6 months
4	3 months

Ashish Maurya

💀 BUT REALITY:

👉 100 dev = 0.12 month ❌ WRONG

👉 Why?

- Communication overhead
- Coordination

👉 This is called:

🔥 **Brook's Law**



3. ESTIMATION TECHNIQUES — DEEP



Empirical (Experience based)

Expert Judgement



Basically:

“Guess based on past”



Problems (VERY IMPORTANT)

1. Bias
 2. Overconfidence
 3. Missing hidden complexity
-



DELPHI TECHNIQUE (VERY IMPORTANT)



Yeh expert judgement ka improved version hai



WHY needed?



Ek expert → biased



Multiple experts → better



Process:

1. Sab experts **individually estimate**
 2. Coordinator collect karta hai
 3. Summary share hoti hai
 4. Experts revise karte hain
 5. Repeat until convergence
-

🔥 Key Insight:

👉 Experts ek dusre ko directly influence nahi karte

💡 IITM Trap:

👉 Delphi ≠ discussion meeting

👉 It is **anonymous + iterative**

🧠 4. HEURISTIC — COCOMO MODEL (CORE 🔥)

? Why COCOMO?

👉 Estimation ko **scientific** banane ke liye

📌 Formula:

Effort = $a \times (\text{SIZE})^b$

🧠 Deep Insight:

👉 Linear nahi hai

👉 SIZE double → effort > double

💡 Reason:

- Complexity grows faster
-

Project Types (VERY IMPORTANT)

1. Organic

- Small
- Experienced team

👉 Example: Simple app

2. Semi-detached

- Mixed team

👉 Example: E-commerce

3. Embedded

- Hardware + software

👉 Example: ATM system

IITM Trap:

👉 Type wrong choose kiya → answer wrong

5. COCOMO EXAMPLE — DEEP THINKING

📌 Page 23–25

Given:

LOC = 4000 = 4 KLOC
Type = Semi

Formula:

Effort = $3 \times (4)^{1.12}$

≈ 15.83 PM

Insight:

👉 Fraction PM allowed hai (approximation)

Cost Calculation

👉 Cost = Effort × salary

Ashish Maurya

6. COST DRIVERS (ADVANCED)

👉 Page 28–30

Why needed?

👉 Real world ≠ ideal

Types:

1. Product

- Complexity

- Reliability
-

2. Hardware

- Memory
 - Speed
-

3. People

- Skill level
-

4. Project

- Tools
 - Schedule pressure
-



Final Formula:

Final Effort = Initial × Adjustment Factor



Example:

$$15.83 \times 1.62 = 25.65 \text{ PM}$$



Key Insight:

👉 Cost drivers effort ko **increase/decrease** karte hain



7. PROJECT SCHEDULING — DEEP

? Scheduling kya hai?

👉 “Kaunsa kaam kab hoga”

🔥 Steps (Page 36 deep meaning)

1. Activities identify

👉 High-level tasks

2. Break into tasks (WBS)

👉 Detailed breakdown

3. Dependencies

👉 Kaun sa task pehle

💡 Example:
Login → Dashboard

4. Time estimation

5. Representation

👉 Graph / Chart

6. Start-End time

7. Critical Path

8. Resource allocation

8. WBS (VERY IMPORTANT)

? Why needed?

👉 Large project ko manageable banane ke liye

Structure:

Project

- Module
- Submodule
- Task

Ashish Maurya

💡 Example:

E-commerce:

- Auth
 - Payment
 - Orders
-

🧠 Insight:

👉 Task size $\approx$ 2 weeks

9. ACTIVITY NETWORK

👉 Graph representation

- Nodes = Tasks
 - Edges = Dependencies
-

🧠 Use:

👉 Critical path find karna

10. CRITICAL PATH (VERY IMPORTANT)

? What?

👉 Longest path

Ashish Maurya

🔥 Why important?

👉 Delay here = full project delay

💡 Example:

$A \rightarrow B \rightarrow C = 10 \text{ days}$

$A \rightarrow D \rightarrow E = 20 \text{ days}$

👉 Critical path = second

11. GANTT CHART

👉 Timeline chart

📌 Page 43–44

Use:

- Visual planning
 - Tracking
-

💀 Limitation:

👉 Dependencies clear nahi hoti

🧠 12. RISK MANAGEMENT — DEEP

? Risk kya hai?

👉 “Future problem ka possibility”

🔥 Types:

1. Technical Risk

👉 Technology related

💡 Example:

- Wrong design
 - Third-party fail
-

2. Project Risk

👉 Management related

💡 Example:

- Delay
 - Budget
-

3. Business Risk

👉 Market related

💡 Example:

- Product fail
-

💀 GOLD PLATING:

👉 Extra features add karna (dangerous)

13. RISK MITIGATION

👉 Risk avoid nahi → reduce

Example:

Wrong UI risk

👉 Solution: Prototype

14. RISK ASSESSMENT (VERY IMPORTANT)

Formula:

Risk = Probability × Impact

Impact levels:

1 = Low

4 = Catastrophic

 Example:

P = 0.6

I = 3

 Risk = 1.8

 Key Insight:

 Highest risk pe focus

15. AGILE PROJECT MANAGEMENT

? Difference?

 Plan vs Agile

Plan

Agile

Fixed

Flexible

Big upfront plan

Iterative

16. SCRUM — DEEP

Sprint

 1–2 weeks

Roles

Product Owner

 Client side

Ashish Maurya

Scrum Master

 Process manager

Dev Team

 Work

17. SCRUM ACTIVITIES

Sprint Planning

👉 Kya + kaise

Daily Scrum

👉 3 questions

- Kal kya kiya
 - Aaj kya karoge
 - Problem kya hai
-

Sprint Review

👉 Demo

Retrospective

👉 Improve

Ashish Maurya

🧠 18. PRODUCT BACKLOG

👉 Prioritized tasks

🧠 Insight:

👉 Sab kuch backlog me hota hai

🧠 19. AGILE ESTIMATION

❌ Problem:

Stories unequal

✅ Solution:

Story Points:

- 1 (easy)
 - 2 (medium)
 - 3 (hard)
-

🔥 Velocity

👉 Points per sprint

💡 Example:

Sprint 1 → 10 points

Sprint 2 → 12

👉 Avg = velocity

🧠 20. FINAL MOST IMPORTANT DIFFERENCE

Plan vs Agile

Feature	Plan	Agile
Estimation	Start	Continuous

Tracking Gantt Velocity
Approach Predictive Adaptive

FINAL MASTER SUMMARY

👉 Estimation:

- LOC, PM
- COCOMO

👉 Scheduling:

- WBS
- Critical path

👉 Risk:

- $P \times I$

👉 Agile:

- Sprint
- Velocity

Ashish Maurya

[Test yourself](#)

WEEK-5

WEEK-5

A 1. Software Design kya hota hai?

👉 Simple language me:

Software Design = system ka blueprint (naksha)

Jaise ghar banane se pehle map banta hai, waise hi coding se pehle design banta hai.

📌 Slides me bhi diya hai:
Design ka output hota hai:

- Components (modules)
- Interfaces
- Data structures
- Algorithms

Ashish Maurya

2. Design Process ka Output

💡 4 important cheezein:

1. Components (Modules)

👉 System ko chhote parts me todna

Example:

- Login module
- Payment module
- Inventory module

📌 Module = functions + data ka group

2. Interfaces

👉 Modules aapas me kaise baat karte hain

Example:

Order module → Payment module ko call karega

3. Data Structures

👉 Data kaise store hoga

Example:

- List
 - Array
 - Database tables
-

4. Algorithms

👉 Kaam kaise hoga (logic)

Example:

- Sorting
 - Searching
 - Payment processing logic
-

3. Good Software Design kaisa hota hai?

Slides me 4 properties di hain 👉

✅ 1. Correctness

👉 System sahi kaam kare

✓ 2. Efficiency

👉 Fast ho + kam memory use kare

✓ 3. Maintainability

👉 Future me easily change ho sake

✓ 4. Understandable

👉 Team ka koi bhi banda samajh sake

🔥 4. Modularity (सबसे important concept)

👉 System ko **modules** me todna

📌 Example:
Amazon Seller Portal:

- Inventory
 - Orders
 - Payments
 - Feedback
-

💡 Why modularity?

- Easy to understand
 - Easy to debug
 - Easy to update
-

🔥 5. Coupling & Cohesion (EXAM KA KING 👑)

🔴 Coupling (Modules ke beech relation)

👉 Kitna ek module dusre pe depend karta hai

- High coupling ❌ (bad)
 - Low coupling ✅ (good)
-

🟢 Cohesion (Module ke andar relation)

👉 Ek module ke andar functions kitne related hain

- High cohesion ✅ (good)
 - Low cohesion ❌ (bad)
-

100 Final Rule:

★ Good Design = High Cohesion + Low Coupling

🔥 6. Types of Coupling

1. Data Coupling ✅ (Best)

👉 Simple data pass hota hai

```
view_inventory(seller_id)
```

2. Control Coupling ❌

👉 Flags pass hote hain

perform_action(flag)

3. Common Coupling ❌

👉 Global variables share hote hain

4. Content Coupling ❌ (Worst)

👉 Ek module dusre ke internal data ko access kare

🔥 7. Types of Cohesion

🟢 1. Functional Cohesion (Best)

👉 Ek module = ek kaam

Example:

- add_item()
 - delete_item()
-

🟢 2. Sequential Cohesion

👉 Output → next input

Example:

create_order → check → place_order

🟡 3. Communicational Cohesion

👉 Same data pe kaam

🟠 4. Procedural Cohesion

👉 Steps follow ho rahe hain but unrelated tasks

🔴 5. Coincidental Cohesion (Worst)

👉 Random functions ek jagah

- **Functional cohesion** → All elements contribute to one well-defined function, but not necessarily in sequence.
- **Coincidental cohesion** → Random grouping of operations with no meaningful relationship.
- **Temporal cohesion** → Operations grouped because they execute at the same time (e.g., initialization tasks).

🔥 1. OOD kya hota hai?

👉 Full form: **Object Oriented Design**

👉 Simple language:

Software ko design karte waqt hum **real world objects** ke hisaab se system banate hain.

💡 Real life example:

Soch tu Amazon bana raha hai:

- Product
- User
- Order
- Payment

👉 Ye sab **objects** ban jaate hain

2. Object kya hota hai?

👉 Object = Real world entity

Example:

- Mobile 📱
 - Car 🚗
 - Student 🎓
-

💡 **Object ke 2 parts hote hain:**

1. Data (Attributes)

👉 Object ki properties

Example:

Product:

name = "Shirt"

price = 999

color = "Blue"

2. Methods (Functions)

👉 Object kya kaam kar sakta hai

Example:

add_product()

delete_product()

update_price()

Important:

★ Object = Data + Methods

3. Encapsulation (VERY IMPORTANT)

👉 Data + Methods ko ek saath pack karna

Simple samajh:

ATM machine:

- Tu sirf button dabata hai
- Andar kya logic chal raha → hidden

👉 Ye hi encapsulation hai

Benefits:

- Security 
 - Code clean hota hai
 - Easy maintenance
-

4. Class kya hoti hai?

👉 Class = Blueprint of object

Example:

Class: Car

Objects:

- BMW

- Audi

- Tesla

👉 Class template hai → object actual cheez hai

Important:

★ Class → object banati hai

5. OOD ka main idea

👉 System ko **objects** ke form me sochna

Instead of:

❌ Functions only

We use:

✅ Objects interacting

6. Class Relationships (Exam me pakka aata hai)

1. Association

👉 Simple relation (help lena)

Example:

Customer → Order

👉 Customer order karta hai

2. Composition (Strong relation)

👉 Whole–Part relation

Example:

Car → Engine

👉 Engine ke bina car useless

3. Inheritance (IS-A relation)

👉 Parent → Child

Example:

Product → Electronics → Mobile

👉 Mobile is a type of electronics

4. Dependency

👉 Ek class dusre pe depend karti hai

Example:

Payment → Bank API

👉 Bank change → payment change

7. Kaise identify kare relationship?

Slides me diya hai 

Association:

 "A uses B"

Composition:

 "A contains B"

Inheritance:

 "A is a type of B"

8. Real Example (Full System)

 Seller Portal (slides wala)

Objects:

- Product
 - Seller
 - Catalogue
-

Example breakdown:

Product class:

Attributes:

- id

- name

- price

Methods:

- add_product()

- delete_product()

Relationships:

- Catalogue contains Product → Composition
- Product → Clothing, Book → Inheritance

📌 Ye diagram page 59 me diya hai



9. Why OOD important?



Advantages:

1. Reusability

👉 Same code reuse

2. Maintainability

👉 Easy to update

3. Scalability

👉 System easily bada hota hai

4. Real world mapping

👉 Easy understanding

10. OOD vs Normal Programming

Feature	Normal	OOD
Focus	Functions	Objects
Structure	Flat	Modular
Reuse	Low	High
Maintenance	Hard	Easy

9. Class kya hoti hai?

👉 Object banane ka template

Example:

Class = Product

Objects = Mobile, Shirt, Book

11. UML (Unified Modeling Language)

👉 Software ka diagram banana

2 Main Views

1. Structural View

👉 System ka structure

Example:

- Class diagram
-

2. Behavioural View

👉 System ka behaviour (time ke sath)

Example:

- Sequence diagram
-

12. Class Diagram

👉 Structure dikhata hai:

- Classes
- Attributes
- Methods
- Relationships

📌 Page 59 me pura diagram diya hai seller portal ka

🔥 13. Sequence Diagram

👉 Step-by-step flow

Example:

User → UI → Controller → Database

📌 Page 61–62 me examples hain

Class diagram → Shows static structure (classes, attributes, relationships).

Sequence diagram → Focuses on **time-ordered interactions/messages** between objects.

Activity diagram → Models workflows and processes, not message exchanges.

Use-case diagram → Captures system functionality from the user's perspective.

🔥 14. UML ka Use kya hai?

✅ **Communication**

Team ko samjhane ke liye

✅ **Development guide**

Code likhne me help

✅ **Code generation**

Diagram → Code

🔥 FINAL COMPARISON (🔥 MUST REMEMBER)

Diagram	Kya dikhata hai	Focus
Class Diagram	Structure	Static

State Machine	States	Status change
Activity Diagram	Flow	Process
Interaction (Sequence)	Communication	Message flow

ULTRA SHORT MEMORY TRICK

- 👉 Class → Structure
- 👉 State → Status change
- 👉 Activity → Flow
- 👉 Sequence → Interaction

FINAL SUMMARY (Revise THIS 👉)

- 👉 Software Design = Blueprint
- 👉 Modular design = Best
- 👉 High cohesion + Low coupling = BEST DESIGN
- 👉 OOP = Objects + Classes
- 👉 UML = Diagram representation

[Test yourself](#)

[Test yourself from week-1 to 5](#) - total 60 mcq questions

WEEK-6

There is no Week-6 in SE

WEEK-7

WEEK-7

1. Introduction to Software Development

👉 (Page 2–4)

💡 **Idea:**

Software banana sirf coding nahi hota — ek **complete process** hota hai:

- Requirements samajhna
- Design banana
- Code likhna
- Test & maintain karna

👉 Is week ka focus: “**Code ke saath kaam kaise karte hain**”

Ashish Maurya

🔧 2. Development Environment (Page 6)

♦ **Kya hota hai?**

Tools ka combination jisse tum code likhte aur run karte ho

♦ **Components:**

1. **Text Editor**
 - Notepad, VS Code, Vim
2. **Compiler / Interpreter**
 - Python, Java runtime

💡 **Example:**

👉 Tum Python code likhte ho VS Code me

👉 Run karte ho Python interpreter se

➡ Ye dono milke = **Development Environment**

3. IDE – Integrated Development Environment (Page 7)

Normal editor vs IDE

Feature	Normal Editor	IDE
Code likhna	✓	✓
Error detect	✗	✓
Auto-complete	✗	✓
Refactoring	✗	✓

Examples:

- VS Code
- PyCharm
- Thonny

Ashish Maurya

 IDE ka kaam: **Developer ki life easy banana**

4. Frameworks (Page 8–9)

Kya hota hai?

Ready-made structure + tools jo development fast karte hain

Example:

 Web app banana hai → Flask use karo

 Tumhe sab kuch scratch se nahi banana padta



5. Flask Web App (Page 10)



Code samjho:

```
from flask import Flask  
app = Flask(__name__)
```

```
@app.route('/')  
def hello_world():  
    return 'Hello World!'
```



Samajh:

- `Flask(__name__)` → app create
- `@app.route('/')` → URL define
- Function → response return

👉 Browser me jaoge → `Hello World!`



6. Templates (Page 11)

```
<h1>Hello {{ username }}</h1>
```



Concept:

Dynamic content generate karna

👉 Example:

- `username = "Ashish"`
 - Output → Hello Ashish
-



7. Interacting with Code (Page 12–13)

♦ 3 main kaam:

1. Write code

2. **Version control (Git)**
3. **Debugging**

👉 Real world me:

- Code likha
 - GitHub pe dala
 - Bug aaya → debug kiya
-



8. Debugging (Page 14–18)

🔥 Definitions (VERY IMPORTANT 🔥)

Term	Meaning
Error	Expected vs actual mismatch
Failure	Visible wrong output
Fault	Code ki galti wali line
Debugging	Error ka root cause find karna

🔄 **Debugging Steps:**

1. Problem reproduce karo
2. Cause find karo
3. Fix plan karo
4. Fix implement karo
5. Test karo

👉 Real Example:

Calculator wrong output de raha → same input repeat → bug track



9. Debugging Techniques (Page 21)

♦ **Methods:**

1. **Logging**
 - print statements
 2. **Dump & diff**
 - compare outputs
 3. **Debugger stepping**
 4. **Profiling**
 - performance check
-

10. Debugging Strategies (Page 22–25)

 **Important (IITM favourite)**

1. Input Manipulation

👉 Input change karo → output observe karo

Example:

N = 2 → [2,3]

N = 6 → [2,3,5,7]

👉 Bug kaha hai? pattern observe karke pakdo

2. Backward Debugging

👉 Output se start → piche jao

3. Forward Debugging

👉 Start se chalo → error tak

4. Blackbox Debugging

11. Python Debugger (PDB) (Page 26–27)

Commands:

```
import pdb; pdb.set_trace()
```

Command	Meaning
---------	---------

n	next line
---	-----------

s	step inside
---	----------------

p	print
---	-------

b	breakpoint
---	------------

c	continue
---	----------

Ashish Maurya



12. Software Metrics (Page 28–32)

♦ Kya hota hai?

Code quality measure karne ka way

👉 Example:

- Code complex hai ya simple?
-



13. Cyclomatic Complexity (Page 33–36)

🔥 Formula:

👉 No. of decisions + 1

◆ Decisions:

- if $\rightarrow +1$
- for $\rightarrow +1$
- while $\rightarrow +1$

Construct	Effect on	
	CC	Reasoning
if	+1	An <code>if</code> statement is a single decision.
elif	+1	The <code>elif</code> statement adds another decision.
else	+0	The <code>else</code> statement does not cause a new decision. The decision is at the <code>if</code> .
for	+1	There is a decision at the start of the loop.
while	+1	There is a decision at the <code>while</code> statement.
except	+1	Each <code>except</code> branch adds a new conditional path of execution.



Example:

```
if x > 0:  
    print(x)
```

👉 Complexity = 2

🔥 Risk Table (Page 36):

CC	Risk
1–5	Low
6–10	Stable
11–20	Moderate
41+	Dangerous ❌



14. Raw Metrics (Page 37)

Metric	Meaning
LOC	Total lines
LLOC	Logical lines
SLOC	Source lines
Comments	Comment counts



15. Halstead Metrics (Page 38)

👉 Advanced metrics (IITM tricky questions 🔥)

Key terms:

- Operators (+, -, etc)
- Operands (variables)

Outputs:

- Program length
- Difficulty
- Effort
- Time
- Bugs

👉 Basically: **Code kitna complex aur error-prone hai**



16. Clean Code (Page 31, 40)

🔥 Kyu important?

- Easy to understand
- Easy debugging

- Easy maintenance
-



17. Code Smells (Page 42–47)

♦ Kya hota hai?

Code me bad practices

🔥 Types:

1. Comments

- ✗ Redundant comments
 - ✗ Commented code
-

2. Functions

- ✗ Too long
 - ✗ Too many arguments
 - ✗ Multiple responsibilities
-

3. DRY (Don't Repeat Yourself)

- ✗ Bad:

```
if month == "April" or month == "June" ...
```

- ✓ Good:

```
months = ["April", "June"]  
if month in months:
```

4. Naming

- ✗ `m = [...]`
- ✓ `thirty_day_months = [...]`



18. Refactoring (Page 48)



Definition:



Code improve karna WITHOUT changing behavior



Tools:

- Pylint
- flake8
- vulture



FINAL REVISION (SUPER SHORT)



Development = tools + frameworks



Debugging = error find + fix



Metrics = quality measure



Complexity = decision count



Clean code = readable code



Code smells = bad patterns



Refactoring = improve without changing output

[Test Yourself](#)

Week-8

WEEK-8

PART 1: Software Architecture (Foundation)

◆ Software Architecture kya hota hai?

👉 Simple language:

Software ka “blueprint” ya “structure” hota hai

📌 PDF (Page 3) ke according:

- Code ko organize karne ka way
- Components/modules define karta hai
- Unke relations bhi define karta hai



Real Life Example

🏠 House banane se pehle:

- Room kaha hoga?
- Kitchen kaha hoga?
- Wiring kaise hogi?

👉 SAME WAY:

Software me bhi architecture decide hota hai

◆ Important Concepts

1 Components (Page 13)

👉 “System ke parts”

Example:

- Client
- Server
- Database

📌 Simple:

Component = “Ek kaam karne wala module”

2 Connectors (Page 14–16)

👉 Components ke beech communication

Example:

- Function calls
- APIs
- REST

📌 Important:

Connector = “kaise baat ho rahi hai components ke beech”

3 Protocols

👉 Rules of communication

Example:

- HTTP
 - REST API rules
-



PART 2: Architecture Styles

◆ 1. Client-Server (Page 5)

👉 Request-Response system

Example:

- Browser → Server

📌 Flow:

Client → Request → Server → Response

◆ 2. Pipe and Filter (Page 6)

👉 Data ek component se dusre me flow hota hai

Example:

- Compiler
- Data processing pipeline

📌 Flow:

Input → Filter → Filter → Output

◆ 3. Peer-to-Peer (Page 12)

👉 No central server

Example:

- Torrent
- Blockchain

📌 Sab nodes equal hote hain

◆ 4. MVC (Model View Controller) (Page 8–11)

🔥 VERY IMPORTANT (Exam fav)

Model

👉 Data handle karta hai

View

👉 UI (user ko kya dikhega)

Controller

👉 User input handle karta hai



Example (Amazon)

Component	Meaning
Model	Product data
View	Website UI
Controller	Button click logic



PART 3: Design Patterns

◆ Design Pattern kya hai? (Page 17)

👉 “Reusable solution”

📌 Matlab:

Common problem → standard solution

◆ Design Smell (Page 18)

👉 Warning ⚠️

Example:

- Duplicate code
 - Huge class
-

◆ Refactoring (Page 20)

👉 Code improve karna without changing behavior

Example:

- Class split karna
 - Functions optimize karna
-

🧠 PART 4: SOLID Principles (🔥 MOST IMPORTANT)

👉 OOP ke rules jo code ko clean banate hain

🔴 S – Single Responsibility Principle (Page 25–26)

👉 Ek class = ek kaam

❌ Wrong:

Seller class + Product + Inventory sab ek me

✅ Correct:

- Seller class
 - Product class
 - Inventory class
-



Memory Trick



“One class = One job”



O – Open Closed Principle (Page 28–30)



Extend karo, modify mat karo



Wrong:

Existing class edit karna



Correct:

Subclass bana do

Example:

- Product → Clothing subclass
-

Ashish Maurya



L – Liskov Substitution Principle (Page 32–35)



Child class parent ki jagah use ho sake



Rule:

Replace karo → system break nahi hona chahiye



Example

EmailNotification → SMSNotification



dono same interface follow kare

I – Interface Segregation Principle (Page 38–42)

👉 “Fat interface” mat banao

✗ Wrong:
Ek interface me sab methods

✓ Correct:
Small interfaces

Example

Delivery system:

Instead of:

- One big interface

Use:

- OnlineDelivery
 - PhysicalDelivery
-

D – Dependency Inversion Principle (Page 44–47)

👉 High-level code → low-level pe depend nahi kare

👉 Interface use karo

**“High-level modules should NOT depend on low-level modules.
Instead, both should depend on abstraction (interfaces)”**



2. Real Life Analogy (Best Samajh 🔥)

❌ Without DIP

👉 Soch:

Tumhara phone **sirf ek specific charger** se hi charge hota hai

👉 Agar charger change ho gaya → phone useless ❌



Example

Payment system:

❌ Direct:
Seller → PayTM

✅ Correct:
Seller → PaymentInterface → PayTM / UPI



PART 5: Design Patterns Types

◆ 1. Creational (Page 52–53)

👉 Object creation

Factory Pattern

👉 Object directly create mat karo

❌ new Courier()

✅ Factory se lo

Factory Example

`courier = CourierFactory.getCourier(type)`

◆ 2. Builder Pattern (Page 60–62)

👉 Complex object banane ke liye

Problem:

Telescoping constructor

`Book(title, author, price, discount, rating...)`

Solution:

Builder pattern

`Book.builder().setTitle().setPrice().build()`

PART 6: Pros & Cons

Factory

- ✓ Flexible
- ✗ Complex code

Builder

- ✓ Step by step creation
- ✗ More classes

Factory vs Builder (Clear Comparison Table)

Feature	Factory Pattern	Builder Pattern
 Purpose	Object creation ko hide/control karna	Complex object step-by-step banana
 Object Complexity	Simple / moderate objects	Complex objects (many fields/options)
 Creation Process	One step me object milta hai	Multiple steps me object banta hai
 Focus	<i>Which object create karna hai</i>	<i>Object kaise build hoga</i>
 Flexibility	Different types return karta hai	Same object ko different ways se build karta hai
 Example	CourierFactory → FedEx/DTDC	BookBuilder → title, price, author set karo
 Runtime Behavior	Type decide karta hai	Structure gradually build hota hai
 Parameters	Usually minimal parameters	Many optional + mandatory parameters
 Encapsulation	Object creation logic hide karta hai	Construction logic separate karta hai
 SOLID Relation	OCP support karta hai	SRP support karta hai

FINAL REVISION (EXAM BOOSTER)

Architecture

- Components + Connectors
- Client-Server, MVC, P2P

SOLID

- S → One job
- O → Extend, don't modify
- L → Replaceable
- I → Small interfaces
- D → Use abstraction

Design Patterns

- Factory → object creation control
 - Builder → complex object creation
-

1. Creational Design Pattern

Kya hota hai?

Object kaise create karna hai — uska pattern

Focus:

 Object creation ko **control + flexible** banana

Ashish Maurya

PART 7: Structural & Behavioral Design Patterns (Page ~65 onward)

Ab tak tumne **Creational** (Factory, Builder) dekh liya

 ab baaki 2 types:

1. Structural Design Patterns

 Focus:

“Classes/objects ko kaise combine kare”

◆ Purpose

- Code ko **flexible & reusable** banana
 - Structure improve karna
-



Real Life Example

👉 LEGO blocks

Tum alag-alag pieces ko combine karke:

- car
- house
- robot

banate ho

👉 SAME idea in software



Common Structural Patterns (conceptual)

1. Adapter Pattern (IMPORTANT)

👉 Do incompatible systems ko connect karta hai

📌 Example:

- Mobile charger → adapter use karke different socket me lagate ho

💻 Software:

- Old API ko new system se connect karna
-

2. Decorator Pattern

👉 Existing object me new functionality add karo

WITHOUT modifying original code

📌 Example:

- Coffee + milk + chocolate
-

3. Facade Pattern

👉 Complex system ko simple interface deta hai

📌 Example:

- ATM → simple UI, andar complex banking system
-

PART 8: Behavioral Design Patterns

👉 Focus:

“Objects ka interaction kaise hota hai”

◆ Purpose

- Communication improve karna
 - Responsibility distribute karna
-

Important Behavioral Patterns

1. Observer Pattern (VERY IMPORTANT)

👉 One-to-many relationship

📌 Example:

- YouTube subscription

Creator → Subscribers notify



Software Example

- Stock price update
 - Notification system
-

2. Strategy Pattern

👉 Different algorithms choose kar sakte ho runtime pe

📌 Example:

- Payment method:
 - UPI
 - Card
 - Net banking
-

3. Command Pattern

👉 Request ko object bana do

📌 Example:

- Remote control buttons
-

PART 9: Comparing Patterns (EXAM FOCUS)

Type	Focus
Creational	Object creation
Structural	Object arrangement

PART 10: When to Use What? (VERY IMPORTANT)

◆ **Factory vs Builder**

Factory	Builder
Simple object creation	Complex object
One step	Step-by-step

◆ **MVC vs Layered thinking**

- 👉 MVC = UI separation
 - 👉 Architecture = overall structure
-

PART 11: Big Picture (MOST IMPORTANT)

Complete Flow

1. Architecture decide karo
 2. SOLID follow karo
 3. Design patterns use karo
 4. Refactor karo
-

👉 Final goal:

Clean, scalable, maintainable system

FINAL MIND MAP (REVISION)

Software Design

→ Architecture

→ MVC, Client-Server, P2P

→ SOLID

→ S O L I D

→ Design Patterns

→ Creational (Factory, Builder)

→ Structural (Adapter, Decorator, Facade)

→ Behavioral (Observer, Strategy)

IITM EXAM TRAPS (LAST PART)

❌ Trap 1:

Factory = reusable object ❌

👉 Factory = object creation control ✓

❌ Trap 2:

Builder = inheritance ❌

👉 Builder = step-by-step construction ✓

❌ Trap 3:

Observer = polling ❌

👉 Observer = push notification ✔️

❌ Trap 4:

Decorator modifies class ❌

👉 It wraps object ✔️

❌ Trap 5:

Facade simplifies UI ❌

👉 It simplifies system interaction ✔️

ULTRA QUICK REVISION (1 MIN)

👉 Structural = structure

👉 Behavioral = interaction

👉 Observer = notify

👉 Strategy = choose algorithm

👉 Adapter = connect incompatible

👉 Decorator = add feature

[Test Yourself](#)

WEEK-9

WEEK-9

1. OVERVIEW (Ye topic kya hai & kyu important hai)

Software Testing kya hota hai?

Software Testing ka matlab:

Program ko check karna ki wo sahi kaam kar raha hai ya nahi

 PDF me bhi diya hai ki:

- choti si error bhi **system crash** kara sakti hai

Real-life example:

- Banking app me bug → paise galat transfer
- Flight system me error → crash tak ho sakta hai

 Isliye testing = **life-saving important step**

Simple samajh:

Coding = ghar banana

Testing = check karna ki ghar gir toh nahi raha 😊

Recap:

- Testing = software ko check karna
- Errors dangerous ho sakte hain
- Isliye testing compulsory hai

2. BASIC TESTING PROCESS

PDF (page 5) ke hisaab se testing ke steps:

Steps:

1. Input do
 2. Output check karo
 3. Expected se compare karo
 4. Error dhundo
 5. Fix karo
 6. Dubara test karo
-

Example:

Tumne ek function banaya:

`add(a,b)`

Test:

- Input: (2,3)
 - Expected: 5
 - Output: 6  → bug
-

Recap:

- Input → Output → Compare → Fix → Repeat
 - Testing ek **cycle** hai
-



3. TEST CASE & TEST SUITE

Test Case kya hota hai?

PDF (page 6):

Test case = [I, S, R]

- I = Input
- S = State (program ki condition)

- R = Expected Result
-

Example:

Function: min(x,y)

Test case:

- Input: (3,5)
 - Expected: 3
-

Test Suite kya hota hai?

Test cases ka collection

Recap:

- Test case = ek check
 - Test suite = multiple checks
-

4. WHY DESIGN TEST CASES?

PDF (page 9):

Random testing se sab errors nahi milte

Example:

Agar tum random numbers test karte ho:

- (5,10)
- (7,20)

But tumne equal case test hi nahi kiya:

- (5,5) ❌ → bug miss
-

👉 Isliye:

- Smart test cases design karne padte hain
-

🔄 Recap:

- Random testing useless hai
 - Proper design zaroori hai
-

● 5. BLACKBOX TESTING

👉 Meaning:

Code ke andar nahi dekhte
Sirf input-output check karte

🧠 Example:

ATM machine:

- Tumhe code nahi pata
 - Bas check: paisa nikal raha ya nahi
-

📌 5.1 Equivalence Class

👉 Meaning:

Inputs ko groups me divide karna

PDF example:

- $x > y$
 - $x < y$
 - $x = y$
-

Example:

Age input:

- valid: 18–60
 - invalid: <18
 - invalid: >60
-

5.2 Boundary Value Analysis

Meaning:

Boundary pe test karo

PDF example:

- isPrime $\rightarrow$ check 0,1
-

Example:

Range: 1–100

Test:

- 0 
 - 1 
 - 100 
 - 101 
-

Recap:

- Blackbox = input-output testing
- Equivalence = groups
- Boundary = edge values

6. WHITEBOX TESTING

Meaning:

Code ke andar jaake testing

Example:

Tum code dekh ke check karte ho:

- if condition
 - loops
 - branches
-



Types of Coverage

1. Branch Coverage

Branch coverage me kya chahiye?

Har decision ke **saare outcomes cover hone chahiye**

if (n < 0) → TRUE + FALSE

for loop → EXECUTE + NOT EXECUTE

 Har branch ek baar execute hona chahiye

Example:

```
if(x<y)
else
```

Test:

- x<y
- x>y

2. Multiple Condition Coverage

👉 Har condition true/false ho

Concept:

- Har **condition** ko TRUE + FALSE dono lena hai
- Yahan sirf **ek condition hai**: $n < 0$

Example:

if(num==0 or num==1)

Test:

- 0
- 1

3. Path Coverage

👉 Har possible path execute ho

PDF example me:

- different paths diye gaye

🧠 Simple Example:

Road map:

- $A \rightarrow B \rightarrow C$
- $A \rightarrow D \rightarrow C$

Dono paths test karne honge

👉 **Branch coverage $\subset$ Path coverage**
(matlab branch coverage, path coverage ke andar aata hai)

◆ **Simple samjho**

- **Branch coverage** → har decision ka TRUE/FALSE check karo
- **Path coverage** → program ke saare possible routes cover karo

👉 **Har path me automatically branches include hote hain**
👉 **Isliye path coverage bada (stronger) concept hai**

Ashish Maurya

📌 **Cyclomatic Complexity**

👉 Formula (PDF page 45):

No. of decisions + 1

🧠 **Example:**

2 if conditions → $2 + 1 = 3$ paths

🔄 **Recap:**

- Whitebox = code-based testing
- Branch, condition, path coverage
- Complexity = number of paths



7. LEVELS OF TESTING

PDF (page 15):



1. Unit Testing

👉 Small parts test karte hain

Example:

- ek function test karna
-



2. Integration Testing

👉 Modules ko jod ke test

Example:

- login + database connect
-



3. System Testing

👉 Full system test

Includes:

- Alpha → company ke andar
 - Beta → users test karte
-



4. Acceptance Testing

👉 Client decide kare accept kare ya nahi

Recap:

- Unit → small
 - Integration → combine
 - System → full
 - Acceptance → client approval
-

8. INTEGRATION TESTING TYPES

1. Big Bang

👉 Sab ek saath integrate

✗ Problem:

- error kaha hai pata nahi
-

2. Bottom-Up

👉 Neeche se start

- Drivers needed
-

3. Top-Down

👉 Top module se start

- Stubs needed
-

4. Mixed

👉 dono ka combination

Recap:

- Big bang risky
 - Top-down & bottom-up better
 - Mixed = practical
-



9. SYSTEM TESTING TYPES



Smoke Testing

👉 Basic features check

Example:

- Login ho raha hai ya nahi
-



Performance Testing

👉 Speed & load test

Example:

- 1000 users ek saath login
-

Recap:

- Smoke = basic check
 - Performance = load check
-



10. TDD (Test Driven Development)

👉 Concept:

Pehle test likho, phir code



Steps (PDF page 62):

1. Test likho → FAIL (RED)
 2. Code likho → PASS (GREEN)
 3. Improve karo → REFACTOR
-



Example:

- Test: add(2,3)=5
 - Code likho
 - Improve karo
-



Recap:

- Test → Code → Improve
 - Agile me use hota hai
-



FINAL SHORT SUMMARY

- ✓ Testing = software check karna
- ✓ Test case = input + expected output
- ✓ Blackbox = without code
- ✓ Whitebox = with code
- ✓ Coverage = kitna code test hua
- ✓ Levels = Unit → Integration → System → Acceptance
- ✓ TDD = pehle test, phir code



1. STUB kya hota hai?



Definition (simple):

Stub = dummy function (fake function)

Jo **missing lower-level module** ki jagah use hota hai



Kab use hota hai?

- Jab **main module ready** hai
- But uske andar jo function call ho rahe hain wo **abhi bane nah**



2. DRIVER kya hota hai?



Definition (simple):

Driver = dummy caller (fake main function)

Jo module ko call karta hai testing ke liye



Kab use hota hai?

- Jab **lower-level module ready** hai
- But usko call karne wala main program nahi bana

[Test Yourself](#)

WEEK-10

WEEK-10

🔥 1. OVERVIEW (Sabse pehle mindset set karte hain)

👉 Software lifecycle yaad hai?

- Requirement → Design → Code → Testing

👉 Week-10 me kya aa gaya?

“Software ready ho gaya... ab kya?”

👉 2 kaam bachte hain:

1. **Deployment** → software ko users tak pahunchana
2. **Monitoring** → check karna ki wo sahi chal raha hai ya nahi

📌 PDF me bhi yehi bola hai

🧠 Real-life analogy (IMPORTANT)

Socho tumne ek restaurant khola 🍴

- Food banana = development
- Taste check = testing
- Restaurant open karna = deployment
- Customers khush hain ya nahi = monitoring

👉 Agar monitoring nahi karoge:

- khana kharab ho sakta hai
 - customers chhod denge
-

🔄 Recap:

- Deployment = “launch”
 - Monitoring = “maintenance + improvement”
-

2. DEPLOYMENT ENVIRONMENTS (Sabse important concept)

👉 Ye exam ka favourite topic hai 🐱

Environment kya hota hai?

Jahan software run hota hai (system + setup)

1. Development Environment

👉 Developer ka personal workspace

📌 PDF page 5: local system, IDE, database

Example:

- Tum VS Code me code likh rahe ho
- apni local DB use kar rahe ho

👉 Problem:

- ye sirf tumhare system pe chal raha hai
 - real world me nahi
-

2. Testing Environment

👉 QA team test karti hai

Example:

- Tumne code diya tester ko
- wo bugs dhundta hai

👉 Yahan:

- alag machine hoti hai
 - real users nahi hote
-

3. Staging Environment (🔥 MOST IMPORTANT)

👉 Ye production ka **exact copy** hota hai

📌 PDF page 7: “exactly resembles production”

? Why needed?

👉 Direct production pe deploy karoge to risk:

- system crash
- users impact

📌 PDF page 9:

configuration errors → live crash

Example:

- Movie release se pehle **final rehearsal**

👉 Yahan:

- real server jaisa setup
 - real testing
-

4. Production Environment

👉 Real world

- users use karte hain
 - paisa yahi banta hai 💰
-

🧠 **Example:**

- Amazon live website
-



IMPORTANT FLOW

👉 Dev → Test → Staging → Production



Recap:

- Dev = coding
 - Test = bug check
 - Staging = rehearsal
 - Production = real world
-



3. DEPLOYMENT STRATEGIES (Kaise deploy karen?)



1. Blue-Green Deployment

👉 2 environments hote hain:

- Blue = current (old)
- Green = new version

Working:

1. New version Green me deploy karo
 2. Test karo
 3. Traffic switch karo
-

Advantage:

- rollback easy

 PDF page 14

Example:

- Flipkart new UI
 - problem → instantly old UI
-

2. Canary Deployment

- 👉 Full users ko nahi dete
 - 👉 thode users ko dete
-

Example:

- Instagram new feature sirf 10% users ko
-

Advantage:

- risk kam
-

Problem:

- multiple versions manage karna padta hai

📌 PDF page 19

3. Versioned Deployment

👉 Multiple versions ek saath

Example:

- App me option:
 - Lite version
 - Full version
-

Recap:

- Blue-Green = instant switch
 - Canary = gradual
 - Versioned = multiple
-

4. HOSTING (Software chalega kaha?)

1. Bare Metal

👉 Apna server kharido

Example:

- apni PC pe website host karna
-

✓ Pros:

- fast
- full control

✗ Cons:

- expensive
 - maintenance
-

☁ 2. IaaS

👉 Infrastructure rent pe

📌 PDF page 26

🧠 Example:

- AWS EC2
-

👉 Tumhe milega:

- virtual machine
- RAM, CPU

👉 baaki sab tum setup karo

🟢 3. PaaS

👉 Platform ready

📌 PDF page 27

🧠 Example:

- Heroku
-

👉 Tum bas code upload karo
👉 baaki sab auto

Recap:

- Bare metal = full control
 - IaaS = partial control
 - PaaS = easy but less control
-

5. CONTINUOUS INTEGRATION (CI)

👉 Problem kya tha?

- multiple devs code likhte
 - conflicts aate
-

👉 Solution = CI

📌 PDF page 31 flow

Steps:

1. Code commit
 2. Auto build
 3. Auto test
 4. Deploy
-

Example:

- Tum GitHub pe push karte ho
 - Jenkins automatically test karta
-

Best Practices:

- daily commit
 - auto testing
 - broken build fix immediately
-

Recap:

- CI = automation
 - fast + safe
-

6. PERFORMANCE (Speed ka game)

1. Caching

 Same data baar-baar DB se mat lao

 PDF page 47

Example:

- Instagram feed cache hota hai
-

2. Task Queue

👉 heavy tasks background me

🧠 **Example:**

- 1000 emails send → background
-

🔴 3. Faster Loading

📌 PDF example:

- slow → users drop
-

Techniques:

- minify code
 - compress files
-

🔄 **Recap:**

- cache → fast
 - queue → efficient
 - optimize → user happy
-



7. MONITORING (Real game yahan hai)

👉 **Monitoring kya hai?**

live system ko observe karna

Metrics:

- CPU
- Memory
- Network
- Storage

 PDF page 53

Example:

- server slow → CPU 100%
-

Tools:

- Lighthouse
-

Recap:

- monitoring = health check
-

8. USER BEHAVIOR (Business side)

Clickstream

User kya click karta hai

Dwell time

kitni der rukta

Abandonment

kahan chhodta

 PDF page 56

Example:

- Cart → Payment → Exit
👉 problem identify
-

Recap:

- user data = gold 💰
-

9. A/B TESTING

Concept:

2 versions compare

Example:

- Button red vs blue
👉 kaunsa better

 PDF page 58

Recap:

- data-based decision



FINAL SUMMARY (SUPER IMPORTANT)

👉 Software lifecycle ka last part:

- Deploy → users tak
- Monitor → improve

👉 Key concepts:

- Environments
 - Deployment strategies
 - CI
 - Performance
 - Monitoring
-



FINAL UNDERSTANDING

“Software banana sirf aadha kaam hai...
asli kaam hai usko smoothly chalana aur continuously improve karna” 🔥

[Test Yourself](#)

WEEK-11

mWEEK-11

🔥 1. OVERVIEW (Ye topic kya hai & kyu important hai)

👉 Ye topic kya cover karta hai?

Is week me coding ya testing nahi — balki:

Software banane ke peeche jo log, communication aur decision-making hoti hai

📌 PDF title bhi same hai

🧠 **Simple samajh:**

Software engineering sirf coding nahi hai ❌

👉 Ye actually hai:

- Teamwork
 - Communication
 - Decision making
 - Organization
-

🧠 **Real-life example:**

Socho Amazon app:

- coder sirf code likhta hai
- par idea kisne diya?
- design kisne banaya?
- user feedback kaun handle karta?

👉 Ye sab milke software banate hain

Recap:

- Coding alone $\neq$ Software Engineering
 - Team + roles + decisions = real engineering
-

2. SOFTWARE ORGANIZATION (Company me kaun kya karta hai)

Goal of Software Company

 PDF page 3:

“Provide value to users”

Different Roles (VERY IMPORTANT)

1. Marketing Team

 Kaam:

- market research
 - user needs samajhna
-

 Example:

- “Log kya chahte hain?”
 food delivery app ya grocery app?
-

2. Product Manager (🔥 MOST IMPORTANT ROLE)

- 📌 PDF me diagram (page 6):
 - 👉 UX + Tech + Business ka intersection
-

👉 Kaam:

- decide **kya banana hai**
 - business + tech + user balance
-

🧠 Example:

- “Flipkart me 1-click buy add kare ya nahi?”
-

⚠ Important:

- coding nahi karta
 - decision leta hai
-

🟣 3. Designers (UX)

👉 Kaam:

- UI/UX design
 - user experience improve
-

🧠 Example:

- button ka color
 - app ka layout
-

🟢 4. Software Engineers

👉 Kaam:

- code likhna

📌 PDF:

engineers decide nahi karte kya banana hai

🧠 **Example:**

- login system implement karna
-

🔴 5. Engineering Manager

👉 **Kaam:**

- teams coordinate karna
 - conflicts solve
-

🧠 **Example:**

- 2 devs fight kar rahe → resolve
-

🟡 6. Sales Team

👉 **Kaam:**

- product sell karna
 - feedback lana
-

🔵 7. Support Team

👉 **Kaam:**

- user problems solve

 Example:

- “App crash ho raha hai” → support help
-

 8. Data Scientists

 Kaam:

- data analyze
-

 Example:

- “Users ka behaviour kya hai?”
-

 9. Ethics & Policy

 Kaam:

- privacy
 - legal rules
-

 Example:

- data safe hai ya nahi
-

 Recap:

- Software = team effort
 - har role important
-

3. COMMUNICATION & COLLABORATION

Concept: Conceptual Integrity

 PDF page 21:

sabko same understanding honi chahiye

Example:

- Dev soch raha login = email
- Designer soch raha login = OTP

 mismatch = problem ❌

Solution:

- proper communication
-

Recap:

- sabka same vision = success
-

4. KNOWLEDGE SHARING TOOLS

Purpose:

 PDF:

knowledge share + store karna

Tools:

◆ **Issue Tracker (JIRA)**

- bugs track
-

◆ **GitHub**

- code + docs
-

◆ **Slack**

- team communication
-

◆ **Stack Overflow**

- problem solving
-

Example:

- bug aya → JIRA me log
 - discussion → Slack
-

Why important?

PDF:

- log chhod dete hain → knowledge chala jata
-

Recap:

- knowledge store karo
 - warna loss ho jayega
-

5. PRODUCTIVITY

Definition:

 “work per unit time”

Important:

 Software me productivity $\neq$ lines of code

Example:

- 1000 lines code 
 - 100 lines efficient code 
-

Tools:

- JIRA $\rightarrow$ track work
 - IDE $\rightarrow$ coding fast
-

Recap:

- smart work > more code
-

🌟 6. WHAT MAKES A GREAT SOFTWARE ENGINEER

👉 Important skills:

- technical knowledge
 - communication
 - teamwork
 - decision making
-

🧠 Example:

- good coder but bad communicator → fail ❌
-

🔄 Recap:

- coding + communication = success
-

🧠 7. DECISION MAKING

📌 Types:

🔵 Macro Decisions

- architecture
 - framework
-

🔴 Micro Decisions

- algorithms
 - code logic
-

Example:

- Macro → React ya Angular
 - Micro → loop ya recursion
-

Process (VERY IMPORTANT)

 PDF page 42

Steps:

1. Problem identify
 2. Options find
 3. Outcomes socho
 4. Best choose
 5. Decision lo
-

Example:

- Database choose:
 - MySQL vs MongoDB
 - compare → decide
-

Recap:

- decision = structured process
-



FINAL SUMMARY

👉 Software engineering = team effort

Key concepts:

- Roles in company
 - Communication
 - Knowledge sharing
 - Productivity
 - Decision making
-



FINAL UNDERSTANDING

“Best software engineers sirf code nahi likhte...
wo communicate, collaborate aur smart decisions lete hain” 🔥

[Test Yourself](#)

Tab 10

COCOMO Project Types

1. Organic Model (Small/Simple)

Used when:

- Small team
- Experienced developers
- Flexible requirements
- Less strict constraints
- Simple business applications

Examples:

- Payroll system
- Library management

Typical size:

- Small projects

Constants:

- $a = 2.4$
 - $b = 1.05$
-

2. Semi-Organic Model (Medium)

Used when:

- Medium complexity
- Mixed experience team
- Some constraints

Examples:

- Compiler
- Database system

Constants:

- $a = 3.0$
 - $b = 1.12$
-

3. Embedded Model (Large/Complex)

Used when:

- Hardware/software tightly coupled
- Real-time constraints
- High reliability required
- Strict regulations
- Very complex systems

Examples:

- Aircraft systems
- Missile systems
- Medical devices
- Embedded systems

Constants:

- $a = 3.6$
- $b = 1.20$

Tab 11

Question 2 : 6406531041138

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

In which of the following types of testing is the fully integrated system tested by a selected group of customers?

Options :

- Integration testing
- Acceptance testing
- Alpha testing
- Beta testing.

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 3 : 6406531041143

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which deployment strategy keeps all the different versions of the production environment live, allowing users to freely choose the version they prefer?

Options :

- Blue/Green Deployment
- Canary Deployment
- Versioned Deployment
- None of these

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 4 : 6406531041144

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which of the following is the major drawback of continuous integration?

Options :

- Long deployment time.
- Initial effort required to set up the process.
- Last-minute confusion and rush at release date.
- Prototypes are available very late in the process.

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 5 : 6406531041145

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Suppose you want to deploy a web application. Your main concern is the highest performance of the web application, and you also want full control of the hardware and software on the server. Which of the following hosting options will be most suitable for you?

Options :

Bare metal servers

Infrastructure-as-a-service

Platform-as-a-service

Software-as-a-service

Your score : 3.000

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 6 : 6406531041146

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which of the following strategies uses tools like Brotli and Gzip to improve the performance of web applications?

Options :

Caching

Task queues

Loading and rendering pages

Monitoring

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 7 : 6406531041148

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

A software organization wants to reuse the knowledge of one project in another project, and it is possible if the knowledge is documented and archived properly. Which kind of tools can be used by the organization?

Options :

Project management tools

Development tools

Knowledge-sharing tools

None of these

Your score : 3.000

Consider the order and delivery modules of an online shopping application. Depending on the products added to the order, the order module passes a flag to the delivery module to indicate whether the order is eligible for express delivery or normal delivery, and that decides which functions are to be invoked from the delivery module. Identify the type of coupling between order and delivery modules.

OPTIONS:

- Data coupling
- Control coupling
- Common coupling
- Content coupling

Question 9 : 6406531041151

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

What are the desired characteristics of a good software design?

Options :

- Low cohesion and low coupling
- Low cohesion and high coupling
- High cohesion and low coupling
- High cohesion and high coupling

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 10 : 6406531041152

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which of the software metrics decides the quality of the code based on the number of lines of the source code?

Options :

- Cyclomatic Complexity
- Raw Metrics
- Halstead's Metrics
- All of these

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 11 : 6406531041153

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Consider debugging a program by controlling its execution flow in a debugger, allowing you to go through the code line by line, inspect variables, and understand the program's behavior in detail.

What type of debugging technique is used in this case?

Options :

- Logging
- Dump and diff
- Stepping in debugger
- Profiling tool

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 12 : 6406531041155

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

In a weather app, users (clients) enter their location to get weather forecasts. The user sends a request to a central system, which processes the request, fetches the weather data from external APIs or databases, and then sends the forecast back to the user. Which type of architecture is used in this weather application?

Options :

[Client-server](#)

[Pipe and filter](#)

[Model-view-controller](#)

[Peer-to-peer](#)

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 13 : 6406531041161

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

What are the risks related to budget, schedule, and staffing commonly known as?

Options :

[Technical risks](#)

[Project risks](#)

[Business risks](#)

[None of these](#)

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 14 : 6406531041163

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

In the Scrum framework, which role is responsible for monitoring all the activities of a sprint and ensuring their successful completion?

Options :

[Product owner](#)

[Scrum master](#)

[Development team](#)

[Client](#)

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 15 : 6406531041167

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

A company developing electronic health records (EHR) software aims to enhance the user interface. To do so, they observe doctors, nurses, and other healthcare professionals in a hospital environment to see how they interact with the current system.

They focus on identifying challenges, such as delays in entering patient data and difficulties in accessing medical histories.

Identify the type of requirement collection technique used in the given scenario.

Options :

Questionnaires

Interviews

Focus Groups

Naturalistic Observations

Your score : 3.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 16 : 6406531041168

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which of the SMART characteristics suggest stopping the implementation of a feature once the time budget is exceeded?

Options :

Measurable

Achievable

Relevant

Timeboxed

Your score : 3.000

Consider a function that takes a string as input. If the length of the string is more than 80, then it prints "Invalid input". If the length of the string is less than or equal to 80, and the string is a palindrome, it returns true; otherwise, it returns false.

Identify the number of equivalence classes for black box testing of the function discussed above.

OPTIONS:

- 2
- 3
- 4
- 5

Question 1

Consider the Python code below.

```
def printPrimes(lower, upper):
    for num in range(lower, upper + 1):
        if num > 1:
            for i in range(2, num):
                if (num % i) == 0:
                    break
            else:
                print(num)
```

Identify the number of linearly independent paths to be tested for path coverage.

OPTIONS:

- 2
 - 3
 - 4
 - 5
-

Question 2

Consider the Python code below.

```
def isEligible(age):
    if age < 0 or age > 100:
        print("Invalid input")
    else:
        if age < 18 or age > 38:
            print("Not eligible")
        else:
            print("Eligible")
```

Identify the set of valid test cases that provides multiple condition coverage.

OPTIONS:

- {age = 10, age = 20, age = 110}
- {age = -10, age = 10, age = 20, age = 110}
- {age = -10, age = 10, age = 20, age = 40, age = 110}
- {age = -10, age = 10, age = 20, age = 30, age = 110}

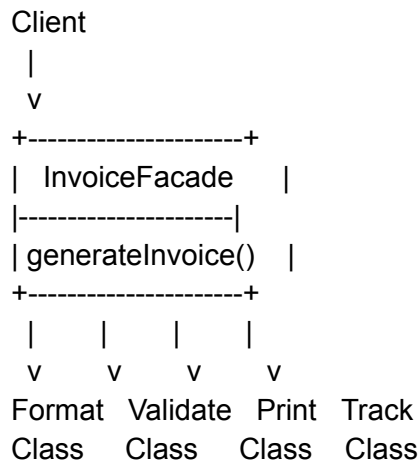
1. Facade Design Pattern (Invoice System)

Idea:

👉 Multiple complex subsystems → ek **simple interface (Facade class)**



Diagram:



Flow:

- Client → sirf `generateInvoice()` call karta hai
 - Facade internally:
 - format karta hai
 - validate karta hai
 - print karata hai
 - track karta hai
-



Key Point:

👉 Complex system hide karke simple interface dena = Facade

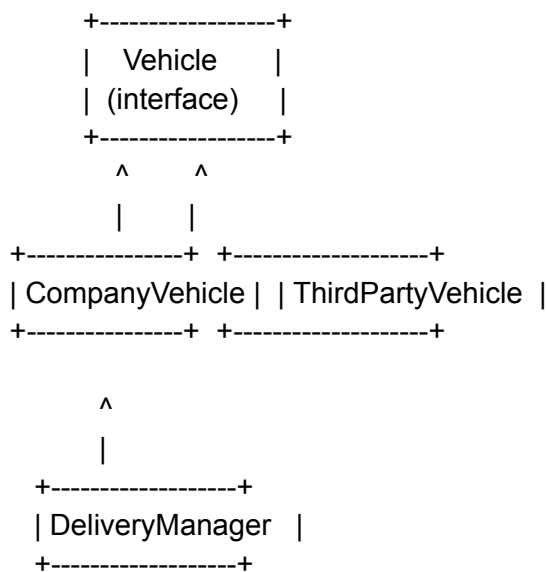
2. Dependency Inversion Principle (Delivery System)

Before (Wrong Design)

DeliveryManager —————> CompanyVehicle

 High-level directly low-level pe dependent

After (Correct Design)



Flow:

- **DeliveryManager** → **Vehicle** interface use karta hai
 - Concrete classes:
 - CompanyVehicle
 - ThirdPartyVehicle
-

Key Point:

👉 Concrete dependency hata ke abstraction introduce kiya

➡ Dependency Inversion Principle (DIP)

Final Answers Recap:

Question	Answer
Invoice system	✅ Facade Pattern
Delivery system	✅ Dependency Inversion Principle

Ultimate Shortcut:

- “single method → multiple internal calls” → Facade
- “class → interface introduce” → DIP

A software company is developing an inventory management system for a pharmaceutical distributor. The system allows its clients (client classes) to generate and print invoices for orders, which involves several steps:

- Formatting the order data into a standardized structure.
- Validating the invoice details.
- Communicating with a third-party printing module to generate the final printed invoice.
- Tracking the status of printed invoices.

A software company is developing an inventory management system for a pharmaceutical distributor. The system allows its clients (client classes) to generate and print invoices for orders, which involves several steps:

Each of these steps requires calling various methods from different classes, which belong to different packages, and must be executed in a very specific sequence. This process became too complex for the developers to handle directly in the main system.

To simplify the interaction, the development team created a new class that provides a unified interface to manage these tasks. This class orchestrates calls to the underlying methods from various classes and packages, enabling the client class to perform all invoice generation related tasks with a single method call.

Which design pattern does this approach represent?

OPTIONS:

- Builder design pattern
- Facade design pattern
- Adapter design pattern
- Strategy design pattern

Consider the following information available for risk assessment of a project.

Risk type	Probability	Assessed risk
Personnel shortfall	0.3	0.9
Gold plating	0.4	0.4
Product not competitive in the market	0.1	0.4
Insufficient domain knowledge	0.5	1.0

Identify the risk type has highest impact on the project.

OPTIONS:

- Personnel shortfall
- Gold plating
- Product not competitive in the market
- Insufficient domain knowledge

Suppose an employee portal lets the HR manager register employee details. The registration form provides validation for each field (e.g., email format, password strength, mandatory fields). It also disables the 'Submit' button until all required fields are correctly filled out. Before submitting the form, a confirmation dialog is displayed to let users verify their inputs.

Which of the following heuristics of UI evaluation is fulfilled in this case?

Options :

Freedom

Flexibility

Prevent error

[Provide help](#)

Your score : 4.000

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 24 : 6406531041165

[View Solutions \(0\)](#) Total Mark : 4.00 | Type : MCQ

An event planning app allows users to organize schedules, send invites, and track RSVPs. However, the app uses a very unique set of icons and labels for its features, which are hard to recall for users who don't use the app frequently. Users often struggle to access key features or complete tasks like editing guest lists or viewing event details after not using the app for a while.

What kind of usability goal is missing from the mobile app?

Options :

[Efficiency](#)

[Memorability](#)

[Safety](#)

[Learnability](#)

Your score : 4.000

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 25 : 6406531041147

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MSQ

Which of the following is/are the primary task/s of a project manager in a software organization?

Options :

[Conduct market research and look for opportunities in the market](#)

[Coordinating between different teams](#)

[Resolving interpersonal conflict between engineers](#)

[Provide feedback to marketing, product, and design teams about the product and its defects/shortcomings](#)

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 26 : 6406531041154

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MSQ

Which of the following statements regarding the Flask framework are correct?

Options :

[It is a framework for developing mobile applications.](#)

[It is a framework for building web applications.](#)

[It is classified as a micro-framework.](#)

[It enforces limitations on application development, such as the types of databases and web servers that can be used.](#)

Your score : 1.5

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 27 : 6406531041166

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MSQ

What is/are the main activities in the maintenance phase of the software engineering process?

Options :

To monitor how users are using the software.

To ensure that the software behaves according to the requirements.

To change the code for upgrades/updates

To generate developer documentation and precise interface definitions

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 28 : 6406531041139

[View Solutions \(0\)](#) Total Mark : 4.00 | Type : MSQ

Which of the following is/are true about designing test cases for whitebox testing?

Options :

Test cases are designed to examine only the input and output values.

Designing test cases necessitates access to the source code.

No knowledge of design or code required

The structure of the program needs to be analyzed using some heuristics.

Your score : 4

Consider the object oriented design of an online music player software. Every user (object of User) can create one or more playlists (objects of PlayList), which comprise a different type of musics (objects of Music). Again, Music can be of several types, like Rock, Jazz, Pop. The design has a MusicPlayer object that plays the musics. The Music class is designed to implement an interface named Playable. Therefore, every change made on the Playable interface results in making changes in class Music. Identify the class relationship(s) which are correct in the above context.

OPTIONS:

- Music is a subclass of MusicPlayer
- Rock, Jazz, and Pop are subclasses of Music
- PlayList is composition of Music
- Playable depends on Player

A small e-commerce company decides to develop a website for selling handmade crafts. The project involves basic functionalities such as product listings, a shopping cart, order

processing, and a payment gateway. The requirements are well understood, and the development team is small and comprises experienced developers.

The values for the constants in the COCOMO model for the different types of projects are as follows:

- Organic project: $a = 2.4$, $b = 1.05$
- Semi-detached project: $a = 3.0$, $b = 1.12$
- Embedded project: $a = 3.6$, $b = 1.20$

A small e-commerce company decides to develop a website for selling handmade crafts. The project involves basic functionalities such as product listings, a shopping cart, order processing, and a payment gateway. The requirements are well understood, and the development team is small and comprises experienced developers.

The values for the constants in the COCOMO model for the different types of projects are as follows:

Based on the above data, answer the given subquestions.

Considering that the source code is 44,000 lines, what is the initial estimated effort for the project?

Options :

128 person-month

208 person-month

338 person-month

181 person-month

Your score : 4.000

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 32 : 6406531041160

[View Parent QN](#) [View Solutions \(0\)](#) Total Mark : 2.00 | Type : MCQ

Considering the average salary of every developer is Rs. 50,000/- per month, what is the initial estimated cost of the project?

Options :

Rs. 7,68,00,000/-

Rs. 64,00,000/-

Rs. 10,40,000/-

Rs. 1,69,00,000/-

Your score : 2.000

Question 2 : 6406531045408

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

In which of the following types of testing is the fully integrated system tested by the test team within the organization?

Options :

Integration testing

Acceptance testing

Alpha testing

Beta testing.

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 3 : 6406531045410

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Consider a function that takes two strings as input and compares the length of two strings. If the lengths of the strings are equal, it returns 0. If the length of the first string is greater than that of the second string, it returns a positive integer. If the length of the first string is less than that of the second string, it returns a negative integer.

Identify the number of equivalence classes for black box testing of the program discussed above.

Options :

2

3

4

5

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 4 : 6406531045413

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which deployment strategy gradually rolls out changes to a small subset of users?

Options :

Blue/Green Deployment

Canary Deployment

Versioned Deployment

None of these

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 5 : 6406531045415

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Suppose you are planning to deploy a data-intensive analytics application. Your main priority is scalability to handle fluctuating workloads, and you prefer a pay-as-you-go pricing model without managing the underlying hardware (but you want control of the software layer). Which of the following hosting options will be most suitable for you?

Options :

Bare metal servers

Infrastructure-as-a-service

Platform-as-a-service

Software-as-a-service

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 6 : 6406531045416

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which of the following strategies uses tools like Memcached and Redis to improve the performance of web applications?

Options :

Caching

Task queues

Loading and rendering pages

Monitoring

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 7 : 6406531045418

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

A software organization wants to use a tool that enables all team members to get both a big-picture view and a detailed view of progress. What kind of tool can the organization use?

Options :

Project management tools

Development tools

Knowledge-sharing tools

None of these

Your score : 0

Consider the inventory and billing modules of an online shopping application. Both modules access and modify a shared global variable called `tax_rate` that is used for calculating taxes on product prices. If the inventory module updates the `tax_rate` whenever there is a change in tax policy, the billing module also uses this shared value for billing calculations.

Identify the type of coupling between the inventory and billing modules.

OPTIONS:

- Data coupling
- Control coupling
- Common coupling
- Content coupling

You have designed software in which each module is responsible for performing varied operations, and a significant amount of data is exchanged between modules.

Identify the types of cohesion and coupling in the designed software.

Options :

Low cohesion and low coupling

Low cohesion and high coupling

High cohesion and low coupling

High cohesion and high coupling

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 10 : 6406531045422

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which software metric determines the quality of the code based on the number of operands and operations involved?

Options :

Cyclomatic Complexity

Raw Metrics

Halstead's Metrics

All of these

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 11 : 6406531045423

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Consider analyzing a program to identify performance bottlenecks by measuring the time taken by each function or the frequency of function calls during execution.

What type of debugging or analysis technique is used in this case?

Options :

Logging

Dump and diff

Stepping in debugger

Profiling tool

Your score : 0

Match the following.

Terms	Definitions
1. Error	A. Observable errors
2. Failure	B. Where the error has occurred
3. Fault	C. Discrepancy between actual behaviors and desired behaviors
4. Debugging	D. Determining the cause of failure

Match the following.

OPTIONS:

- 1-B, 2-D, 3-C, 4-A
 - 1-B, 2-D, 3-A, 4-C
 - 1-C, 2-A, 3-D, 4-B
 - 1-C, 2-A, 3-B, 4-D
-

Which of the following risks arises when the development team has insufficient knowledge of the product?

Options :

Technical risks

Project risks

Business risks

None of these

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 14 : 6406531045433

View Solutions (0) Total Mark : 3.00 | Type : MCQ

In the Scrum framework, what does velocity mean?

Options :

Number of user stories completed per iteration/sprint

Number of user stories completed for all the sprints

Number of points per iteration/sprint

Amount of effort for each user story

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 15 : 6406531045438

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MCQ

Which SMART characteristic specifies that a user story should provide business value to at least one stakeholder?

Options :

Measurable

Achievable

Relevant

Timeboxed

Your score : 0

Consider the Python code below.

```
def countDivisorsInRange(lower, upper):  
    for num in range(lower, upper + 1):  
        divisor_count = 0  
        for i in range(1, num + 1): # Loop to find divisors of `num`  
            if num % i == 0:  
                divisor_count += 1  
        print(f"{num} has {divisor_count} divisors.")
```

Consider the Python code below.

Identify the number of linearly independent paths to be tested for path coverage.

OPTIONS:

- 2
 - 3
 - 4
 - 5
-

Consider the Python code below.

```
if len(password) < 3 or len(password) > 8:  
    print("invalid password")  
else:  
    if (len(password) > 5):  
        print("strong password")
```

```
else:  
    print("weak password")
```

Consider the Python code below.

Identify the set of valid test cases that provides multiple condition coverage.

OPTIONS:

- {password = "ab", password = "abcd", password = "abcdef"}
- {password = "ab", password = "abcdefghi", password = "abcdef"}
- {password = "ab", password = "abcdefghi", password = "", password = "abcd"}
- {password = "ab", password = "abcdefghi", password = "abcdef", password = "abcd"}

In a data processing application, raw sensor data is collected and passed through a series of stages: first, the data is cleaned, then it is normalized, after which statistical analyses are performed, and finally, visualizations are generated. Each stage processes the output from the previous stage and passes it to the next.

Which type of architecture is used in this data processing application?

Options :

Client-server

Pipe and filter

Model-view-controller

Peer-to-peer

Your score : 0

A financial software company is developing a portfolio management system that calculates the returns on investment (ROI) for its clients based on different investment methods. Each client can choose a method for calculating ROI, such as:

- Fixed Deposit Method: Calculates ROI using fixed interest rates over time.
- Stock Market Method: Calculates ROI based on fluctuating market prices and dividends.
- Real Estate Method: Calculates ROI based on property appreciation and rental income.

The calculation logic for each method varies significantly. Instead of hardcoding these calculation methods directly into the portfolio management system, the development team decides to create a flexible design. They define a common interface for ROI calculation, and each method implements this interface with its specific calculation logic.

The portfolio management system dynamically selects and applies the appropriate calculation method based on the client's chosen investment type, enabling the system to easily accommodate new methods in the future without modifying existing code.

Which design pattern does this approach represent?

OPTIONS:

- Builder design pattern
 - Facade design pattern
 - Adapter design pattern
 - Strategy design pattern
-

In a content management system, the **ContentManager** class interacts with various types of content: **Article**, **Video**, and **Podcast**. Initially, all types of content implement a single interface **Content**, which includes methods for actions like **play()**, **download()**, and **share()**.

However, it is later discovered that not all types of content support all actions. For example, **Article** objects do not support **play()**, and **Podcast** objects do not support **download()**.

To address this, the system designer decides to split the **Content** interface into smaller, more specific interfaces: **Playable**, **Downloadable**, and **Shareable**. Now, each content type implements only the interfaces that correspond to its capabilities.

What kind of SOLID principle is applied to improve the design of the content management system?

OPTIONS:

- Open-Closed principle
 - Liskov substitution principle
 - Interface segregation principle
 - Dependency inversion principle
-

Consider the following information available for risk assessment of a project.

Risks	Impact	Assessed risk
Schedule Slippage	3	0.9

Gold plating	1	0.4
Shortcomings in external components	3	0.6
Insufficient domain knowledge	2	1.0

Consider the following information available for risk assessment of a project.

Which of the following risks is most probable to occur in the given project?

OPTIONS:

- Schedule Slippage
 - Gold plating
 - Shortcomings in external components
 - Insufficient domain knowledge
-

Consider a project management tool that allows users to create and manage tasks. The interface provides multiple ways to interact with the system: users can add tasks via a detailed form, use drag-and-drop functionality to create tasks on a visual board, or use keyboard shortcuts for quick task entry. Advanced users can also customize their workspace layout and save presets for different project types.

Which of the following UI evaluation heuristics is demonstrated in this scenario?

Options :

Freedom

Flexibility

Prevent error

Provide help

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 23 : 6406531045435

[View Solutions \(0\)](#) Total Mark : 4.00 | Type : MCQ

An event planning app allows users to organize schedules, send invites, and track RSVPs. However, the app uses a very unique set of icons and labels for its features, which are hard to recall for users who don't use the app frequently. Users often struggle to access key features or complete tasks like editing guest lists or viewing event details after not using the app for a while.

What kind of usability goal is missing from the mobile app?

Options :

Efficiency

Memorability

Safety

Learnability

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 24 : 6406531045437

[View Solutions \(0\)](#) Total Mark : 4.00 | Type : MCQ

A startup developing a fitness center management platform studies daily gym operations to understand how staff manage memberships, schedule personal training sessions, and track equipment usage. The goal is to identify issues such as overlapping schedules, missed booking notifications, or difficulties updating member profiles. Identify the type of requirement collection technique used in the given scenario.

Options :

Questionnaires

Interviews

Focus Groups

Naturalistic Observations

Your score : 0

Discussions (0)[Suggest Correction](#)[Broken Format?](#)

Question 25 : 6406531045409

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MSQ

Which of the following is/are true about designing test cases for blackbox testing?

Options :

Test cases are designed to examine only the input and output values.

Designing test cases necessitates access to the source code.

No knowledge of the underlying design or code required

The structure of the program needs to be analyzed using some heuristics.

Your score : 0

Which of the following are the key tasks in a continuous integration server?

OPTIONS:

- Developers commit the code to a continuous integration server.
- After each commit, the continuous integration server pulls the new code from the version control system.

- The code pulled from the version control system is built and tested on the continuous integration server.
- The code is finally built, tested, and deployed by the continuous integration server.

Which of the following is/are the primary task/s of sales team in a software organization?

Options :

Conduct market research and look for opportunities in the market

Sell the product to users that marketing team has identified

Resolving interpersonal conflict between engineers

Provide feedback to marketing, product, and design teams regarding the product, which engineers then address.

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 28 : 6406531045436

[View Solutions \(0\)](#) Total Mark : 3.00 | Type : MSQ

In today's world, where software development occurs in distributed environments, it is crucial for developers to create precise interface definitions. Based on the provided options, which statement(s) regarding interface definitions is/are correct?

Options :

It describes the capabilities of a function.

It provides the implementation details of a function.

It tells which requests are allowed and how the output will be formatted.

The code implementing the interface should remain unchanged until the interface remains the same.

Your score : 0

Consider the object-oriented design of an e-commerce platform. Each user (object of Customer) can have one or more orders (objects of Order), where each order contains multiple items (objects of Item). An Item can belong to various categories, such as Electronics, Clothing, and Books. The design includes a Cart object that allows customers to manage items before placing an order.

The Item class is designed to implement an interface called Purchasable. Therefore, any change made to the Purchasable interface will require corresponding changes in the Item class.

Identify the correct class relationship(s) in this context.

OPTIONS:

- Item is a subclass of Order
- Electronics, Clothing, and Books are subclasses of Item
- Order is composition of Item
- Purchasable depends on Item

A company has been assigned the development of a smart home controller system. This controller is designed to centralize and integrate multiple devices, including lighting, HVAC systems (Heating, Ventilation, and Air Conditioning), and security cameras. The project requires creating both hardware-specific drivers and application-level software to enable smooth and efficient operation.

The values for the constants in the COCOMO model for the different types of projects are as follows:

- Organic project: $a = 2.4$, $b = 1.05$
- Semi-detached project: $a = 3.0$, $b = 1.12$
- Embedded project: $a = 3.6$, $b = 1.20$

A company has been assigned the development of a smart home controller system. This controller is designed to centralize and integrate multiple devices, including lighting, HVAC systems (Heating, Ventilation, and Air Conditioning), and security cameras. The project requires creating both hardware-specific drivers and application-level software to enable smooth and efficient operation.

The values for the constants in the COCOMO model for the different types of projects are as follows:

Based on the above data, answer the given subquestions.

Considering that the source code is 1,00,000 lines, what is the initial estimated effort for the project?

Options :

303 person-month

208 person-month

522 person-month

905 person-month

Your score : 0

Discussions (0) [Suggest Correction](#) [Broken Format?](#)

Question 32 : 6406531045430

[View Parent QN](#) [View Solutions \(0\)](#) Total Mark : 2.00 | Type : MCQ

Considering the effort adjustment factor is 1.62, what is the final estimated effort for the project?

Options :

1467 person-month

491 person-month

524 person-month

907 person-month

Your score : 0